Monash University

FIT5202 – Data Processing for Big Data 2026 S2

Assignment 2A: Building ML Models for Smart Transport & Predictive Parking

Due: **(23:55 Friday 16/October/2026 End of Week 11)**
Worth: 15% of the final marks

Background

Urban congestion and parking search inefficiencies represent major challenges for modern smart cities. Studies show that up to 30% of downtown traffic congestion is caused by drivers cruising to find an open parking bay. Traditional smart parking systems suffer from the "stale parking" dilemma: displaying available parking spaces at the moment of search is of limited utility to a motorist still minutes away in traffic, as those vacant bays will likely be occupied before the driver arrives.

To address this challenge, metropolitan transport authorities are transitioning towards integrated, predictive smart navigation systems. By coupling historical road traffic volume, corridor transit performance, and vehicle fleet characteristics with IoT parking sensor data, transport authorities can anticipate future parking pressure. Accurate forecasting allows municipal dispatchers to implement dynamic traffic calming, advise drivers on arrival-time availability, and optimise urban mobility.

Big data architectures and distributed machine learning let us analyse vast multi-source transport datasets, handle spatiotemporal correlations, isolate meaningful predictive features from sensor noise, and build models capable of scalable inference.

Objective of the Project

In this assignment, you will act as a lead data scientist for a municipal smart transport agency (such as Transport Victoria or the City of Melbourne). In Assignment 1, you explored large-scale distributed batch processing and spatiotemporal joins on historical mobility records. In Assignment 2 (parts A and B), you will transition from historical analysis to predictive machine learning and real-time streaming processing:

- **Assignment 2A (This Assignment):** You will use Apache Spark's MLlib to construct,

optimise, evaluate, and persist machine learning pipelines. Your models will learn the non-linear relationship between corridor traffic congestion, temporal patterns, vehicle attributes, and parking space availability.

- **Assignment 2B:** You will utilise Apache Spark Structured Streaming, Apache Kafka, and your persisted ML pipeline from Assignment 2A to process live sensor telemetry, making real-time arrival and parking availability forecasts displayed on an interactive dashboard.

The Datasets:

The datasets can be downloaded from Moodle as a compressed archive (A2A_SmartTransport_Data.zip). The data represents an integrated urban transport environment combining municipal IoT sensors, traffic loop counters, and vehicle registry data:

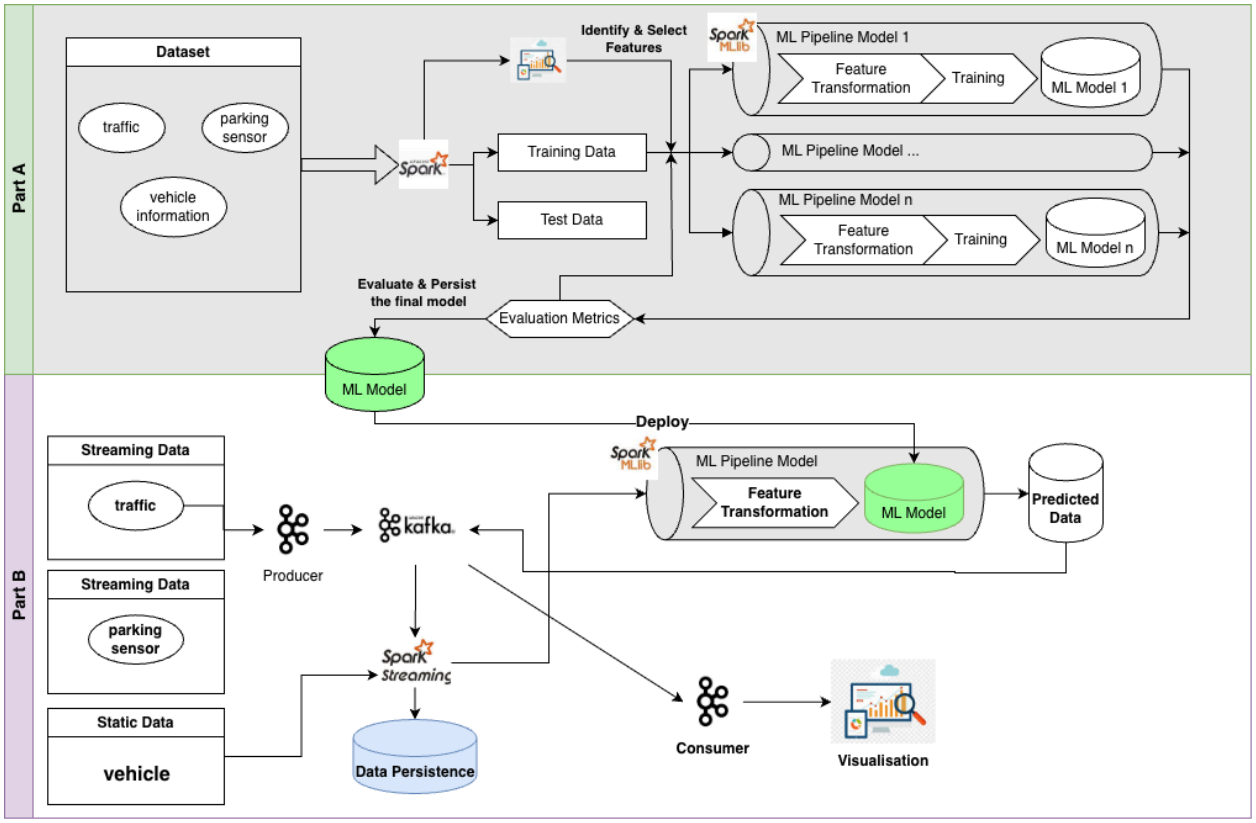
1. a2_traffic_data.csv: Contains 15-minute interval-based aggregations of traffic volumes, traffic direction, and vehicle class.
2. a2_parking_sensor.csv: Contains parking bay information powered by IoT sensors.
3. vehicle_types.xlsx: A data sheet containing vehicle profiles and classification.
4. area.json and street.json: Complete location information about parking bays, similar to assignment 1.
5. A2a_sites.csv: Contains latitude and longitude of traffic sensors.
6. Metadata (in appendix): Contains metadata/schema information for the dataset.

Note on Data Realism: These datasets have been curated to reflect real-world operational environments. Consequently, they contain redundant metadata, sensor diagnostic fields, and auxiliary noise columns that are deliberately non-predictive. You are expected to identify and filter these attributes during feature selection.

Note on Specification Clarification and Update: Students must check the Ed Forum regularly for official clarifications and specification updates.

What You Need to Achieve

Component	Description	Approach
Use Case	Predict destination parking availability (available_spots) conditioned on spatiotemporal traffic congestion and urban mobility features.	Supervised Regression



In both Assignment 2A and 2B, all data processing, feature engineering, and model building must be implemented using **PySpark DataFrames** and **PySpark MLlib**. Pandas is strictly permitted only for local post-aggregation plotting and summary tables.

Getting Started

1. Download the dataset archive and the starter template notebook (A2A_template.ipynb) from Moodle.
2. Rename your notebook file according to your student ID: A2A_<authcateID>.ipynb (e.g., A2A_jdoe0001.ipynb).
3. Ensure your environment matches the course standard: Python 3+ and PySpark 4.1.0+.
4. Answer each section directly within your notebook using markdown and code cells. Clearly cite any external resources or Generative AI prompts used.

Part 1: Data Loading, Transformation, and Exploration (30%)

1.1 Data Loading (2%)

1. **SparkSession Configuration:** Write the code to initialise a SparkSession. You must use a SparkConf object to set a descriptive application name, ensure the maximum partition byte size does not exceed **32MB**, and configure Spark to execute locally using all available CPU cores.
2. **Schema Definition:** Explicitly define StructType schemas for a2_traffic_data.csv,

a2_parking_sensor.csv, adhering to the data types specified in metadata.xlsx. Do not rely on schema inference such as inferSchema.

3. **DataFrame Ingestion:** Load the CSV files into distinct PySpark DataFrames using your defined schemas and print their schemas to confirm correct types.

1.2 Data Transformation and Feature Creation (14%)

Background and domain knowledge before you explore and join the DataFrames:

a) The a2_traffic_data dataset contains sensor readings with aggregated counts for various vehicle types along with their travel direction. Depending on the traffic direction and flow towards the central business district (CBD) parking availability may be closely correlated with these traffic movement patterns.

b) Vehicle type may be correlated with parking; for instance, some trucks are just bypass the city and won't park there.

c) Workers working in different area/zone may go to work and park there at different time. We can try to discover the pattern using Spark MLlib.

d) Depending on the traffic sensor location, a car travelling on the M1 at 7am may occupy a parking spot at 8 am.

Tasks:

1. **Spatial-Temporal Join & Aggregation:** Based on the domain knowledge discussed above, develop a join strategy to join the traffic counts and parking availability data frames, then write PySpark code to join the traffic and parking datasets. Your resulting consolidated DataFrame must isolate available_spots as the continuous numerical value you will use as the target label.
2. **Imputation:** Sensor malfunctions and transmission dropouts frequently create missing records in municipal IoT feeds. Identify at least **two numerical features** with missing values (null or recorded invalid placeholders). Use the **Spark MLlib Imputer (refer to: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.Imputer.html)** (or implement a domain-justified conditional imputation strategy) to fill these missing values. Justify why your chosen strategy (mean, median, or other modelling) is mathematically appropriate for each attribute.
3. **Feature Engineering:**

- *Temporal Congestion Window:* Urban parking availability exhibits strong cyclical dependency on commuter behaviour. Extract the hour and day_of_week from the timestamp and derive a categorical feature Peak_Period (e.g., classifying weekday morning rush 07:00–8:59 and evening rush 15:00–17:59 as Peak and remaining intervals as Off-Peak).
- *Domain Feature Creation:* Create at least **one additional engineered feature** combining multiple attributes (for example: an estimated *Traffic Pressure Ratio* dividing traffic volume by total zone parking capacity, an interaction between traffic volume and parking availability, or an aggregated commercial heavy vehicle index). Explain the domain intuition behind your feature.

1.3 Exploratory Data Analysis (14%)

1. **Statistical Summary:** Using your prepared DataFrame, generate a structured statistical overview:
   - For all numerical features, output count, mean, standard deviation, min, max, 25th percentile, 50th percentile (median), and 75th percentile. (Note: numerical features do not include IDs, just those columns related to the analysis.)
   - For categorical/string features, display the top 5 most frequent categories along with their counts.
2. **Bivariate and Multivariate Visualisations:** Write code to produce **two distinct bivariate or multivariate plots** analysing interactions within the data (e.g., *Traffic volume vs. Available Parking Spots*, or *Average Speed vs. Occupancy Rates during Peak vs. Off-Peak Hours*).
   - Provide an analytical discussion beneath each figure (maximum 150 words per plot) detailing the observed trend and its practical implications for model training.

Part 2: Feature Extraction and ML Training (35%)

2.1 Feature Selection & Justification (10%)

Review the columns across the joined datasets. In a markdown cell, provide a structured discussion (maximum 300 words):

- Identify which specific columns represent **noise or distractors** for the target label and explain why they should be excluded.
- Justify the predictive features you have selected for your feature vector.
- Detail the transformations required (e.g., index encoding, one-hot encoding, vector scaling) before vector assembly.

Write the PySpark DataFrame code to assemble your filtered feature set.

2.2 Preparing Spark ML Pipelines (10%)

Construct the required data transformation stages using PySpark MLlib:

- StringIndexer and OneHotEncoder for non-ordinal categorical features.
- VectorAssembler to combine all numerical and encoded categorical columns into a single features vector.
- Perform scaling and normalisation as appropriate.

(Note: If you have already created those transformations, you can use them directly in the pipelines.)

Define two ML Estimators configured for continuous label regression:

- **Pipeline 1:** RandomForestRegressor
- **Pipeline 2:** GBTRegressor (Gradient-Boosted Tree Regressor)

Assemble each estimator with your feature transformation stages into two clean, end-to-end pipeline objects. **Do not call .fit() or .transform() at this stage**.

2.3 Model Training and Evaluation (15%)

1. **Data Splitting:** Split your consolidated dataset into training (70%) and testing (30%) subsets using seed=2026.
2. **Metric Discussion:** Discuss the evaluation metrics you will use to evaluate your regression models (e.g., Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), R^2, etc.). Explain why you chose those metrics over others.
3. **Pipeline Fitting & Comparison:** Train both pipelines on the training split. Evaluate their performance on the unseen test split using RegressionEvaluator. **Present a side-by-side comparison table of the evaluation metrics**, analyse why one algorithm outperformed the other, and save the superior pipeline model to disk. (You will load this model in Assignment 2B).

Part 3: Performance Tuning (35%)

Using the superior model pipeline identified in Part 2, perform systematic hyperparameter tuning to optimise predictive accuracy.

1. **Search Grid & Cross-Validation:** Construct a parameter grid, and select at least **two hyperparameters** to tune (e.g., maxDepth, numTrees/maxIter, or stepSize/minInstancesPerNode).
2. Configure a CrossValidator with either k-fold cross-validation or TrainValidationSplit, using the metrics you suggested as the primary optimisation metric.
3. **Process Documentation & Analysis:**
   In a markdown cell, document:
   - The parameters and value ranges tested in your grid.
   - The computational cost and observed convergence behaviour across folds.
   - The quantitative performance gain achieved relative to your baseline model from Part 2.
4. **Model Export:** Save your finalised, tuned pipeline model to an export directory (e.g., models/best_tuned_parking_model). This persisted model will be directly deployed into your real-time Kafka streaming consumer in Assignment 2B.

Submission Guidelines

Submit your final assignment solution via Moodle. You must submit the following files. Both files are required for marking; failure to submit either file may incur a penalty of up to 10%.

1. Your Jupyter notebook file **(e.g., A2A_authcate.ipynb**).
2. **A PDF file** saved from jupyter notebook with **ALL** output following the file naming format as follows: **A2A_authcate.pdf**
3. You **don't** need to submit your trained model.

Important: All outputs must be visible in both the submitted notebook and PDF. Code without its corresponding output may lose mark, unless otherwise specified.

Note that all submitted files will be scanned using plagiarism detection software. The highest similarity score among students may be asked to prove the originality of the work.

Assignment Marking Rubric

For each task individually, you'll be marked based on the quality of your work on a 3-level scale (0%, 50% and 100%).

1. 0%: No attempt or incorrect answer with poor attempt;
2. 50%: Partial mark for a good attempt but incorrect result;
3. 100%: Full mark for a correct attempt.

In your submission, the Jupyter notebook file should contain the **code and its output**. It should follow *programming standards, code readability, and code organisation*. Please find the PEP 8 -- Style Guide for Python Code for your reference. Here is the link: https://peps.python.org/pep-0008/ **A penalty of up to 10% applies if your code is hard to understand due to insufficient comments.**

Late submissions and Special Consideration

All special consideration, including within the semester, is now to be submitted centrally. This means that students MUST submit an online Special Consideration form via Monash Connect. For more details, please refer to the **Unit Information** section in Moodle.

A late submission is subject to a 5% penalty per day, including weekends. The cut-off date is 7 days after the due date. No submission (i.e. 0 marks) will be accepted after the cut-off date unless you have a special consideration.

If you submit the wrong file and ask for the submission to be reopened, the new resubmission timestamp will be used to calculate the late penalty.

Mark Release and Review

1. Mark will be released within 10 business days after the submission deadline. For assessment with demo/interview, the "submission deadline" refers to the date when the majority of the interview is completed; not the due date in Moodle.
2. Reviews and disputes regarding the mark will be accepted a maximum of 7 days after the release date (including weekends).

Other Information

Where to get help

You can ask questions about the assignment in the Assignments section in the Ed Forum accessible on the unit's Moodle Forum page. This is the preferred venue for assignment clarification-type questions. Students are expected to check the Ed Forum regularly, as responses from the teaching staff are considered official and may constitute amendments or additions to the assignment specification. Clarifications, updates, and additional information will be posted there to help avoid confusion when completing the assignment. **Failure to check the forum will not be accepted as a reason for misunderstanding the assignment requirements.**

Also, you can attend scheduled consultation sessions if the problem and confusion are still unresolved.

Plagiarism and collusion

Plagiarism and collusion are serious academic offences at Monash University. Students must not share their work with any other students. Students should consult the policy linked below for more information.

https://www.monash.edu/students/academic/policies/academic-integrity

See also the video linked on the Moodle page under the Assignment block.

Students involved in collusion or plagiarism will be subject to disciplinary penalties, which can include:

- The work is not being assessed
- A zero grade for the unit
- Suspension from the University
- Exclusion from the University

Generative AI Statement

As per the University's policy on the guidelines and practices pertaining to the usage of Generative AI:

AI & Generative AI tools may be used SELECTIVELY within this assessment.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

1. Represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
2. Adhere to the AI use guidelines set for the assessment task.
3. Reflect the University's commitment to academic integrity and ethical behaviour.

Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

The teaching team encourages students to apply their own critical thinking and reasoning skills when working on assessments, with assistance from GenAI. Generative AI tools may produce inaccurate content, which could negatively impact students' comprehension of big data topics.

Data source acknowledgement:

The dataset is a remix based on several real-world datasets and synthetic data.

Appendix: Metadata

a2a_traffic_data

The `a2a_traffic_data` dataset contains sensor readings and aggregated traffic measurements recorded at various monitoring sites.

Column Name	Data Type	Description
date	Date	The date on which the traffic sensor readings were recorded.
time_bin	String	The specific time interval or time slot during the day for the aggregated reading (e.g., 15-minute blocks).
site	Integer	Unique identifier for the physical sensor location or traffic monitoring station.
heading	String	The directional movement of vehicles passing the sensor (e.g., Northbound, Southbound, Eastbound, Westbound).
vehicle_class	Integer	The classification category of the observed vehicles (e.g., Passenger Cars, Heavy Commercial Vehicles, Buses, Motorcycles).
speed_bin	String	The speed bracket or range in which the vehicles were detected traveling (e.g., 0–20 km/h, 20–40 km/h, >60 km/h).
volume	Integer	The aggregated count/number of vehicles recorded for the given combination of date, time bin, site, heading, vehicle class, and speed bin.
id	Integer	Unique record/row identifier for each observation entry in the dataset.

Column Descriptions & Analytical Utility

- **Temporal Features (`date`, `time_bin`):** Allow for time-series aggregation to examine peak vs. off-peak traffic flow patterns and weekly variations.
- **Spatial & Directional Features (`site`, `heading`):** Identify traffic flow volume moving toward specific destinations (e.g., inbound towards the CBD), which directly relates to parking demand and availability patterns.
- **Vehicle & Speed Attributes (`vehicle_class`, `speed_bin`, `volume`):** Provide detailed insights into traffic composition, congestion levels, and flow dynamics.
- **Identifier (`id`):** Ensures primary key integrity across data processing pipelines and relational joins.

a2a_parking_sensor

The `a2a_parking_sensor` dataset captures real-time and historical vehicle occupancy events recorded by on-street parking sensors, detailing vehicle arrivals, departures, parking restrictions, and location metadata.

Column Name	Data Type	Description
deviceid	Integer	Unique identifier for the physical parking sensor device installed in the bay.
arrivaltime	Timestamp	Date and time when a vehicle entered the parking bay.
departuretime	Timestamp	Date and time when the vehicle exited the parking bay.
streetmarker	String	Unique physical street marker identifier associated with the parking bay location.
signplateid	Integer	Identifier for the parking rule signplate governing the bay regulations.
sign	String	Text description of the parking sign restrictions (e.g., time limits, fee requirements, permit rules).
streetid	Integer	Unique identifier for the street where the parking bay is located.

Column Name	Data Type	Description
betweenstreet1id	Integer	Identifier for the first intersecting cross street defining the street block segment.
betweenstreet2id	Integer	Identifier for the second intersecting cross street defining the street block segment.
sideofstreet	Integer	Numerical code indicating the specific side of the street where the bay is positioned.
sidename	String	Cardinal or ordinal orientation of the street side (e.g., North, South, East, West).
bayid	Integer	Unique identifier for the individual parking bay.
inviolation	Boolean	Flag indicating whether the parking event exceeded signposted time limits or violated parking regulations.
vehiclepresent	Boolean	Status indicator of vehicle presence/occupancy in the bay at the time of the event reading.
areaid	Integer	Identifier for the broader geographic or administrative parking precinct/zone.
d_year	Integer	Four-digit year derived from the event timestamp for partitioned querying.
d_month	Integer	Month number (1–12) derived from the event timestamp.
d_day	Integer	Day of the month (1–31) derived from the event timestamp.
d_minute	Integer	Minute of the hour (0–59) derived from the event timestamp.
d_hour	Integer	Hour of the day (0–23) derived from the event timestamp.

Column Descriptions & Analytical Utility

- **Duration & Occupancy Analysis (`arrivaltime`, `departuretime`, `vehiclepresent`):** Enables computation of stay durations, bay turnover rates, and average occupancy levels across peak and off-peak periods.
- **Spatial & Geographic Segmentation (`bayid`, `streetid`, `betweenstreet1id`, `betweenstreet2id`, `areaid`, `streetmarker`):** Facilitates micro-level spatial mapping to identify high-demand blocks, precinct utilization, and parking hotspot trends.
- **Compliance & Policy Monitoring (`inviolation`, `signplateid`, `sign`):** Evaluates overstay frequency and compliance rates against specific parking restrictions to guide enforcement allocation and policy adjustments.
- **Temporal Partitioning & Time-Series Analysis (`d_year`, `d_month`, `d_day`, `d_hour`, `d_minute`):** Provides optimised data partitioning keys for efficient temporal querying and trend modelling across daily, monthly, and seasonal cycles.